

PHP — Orientação a Objetos

Herança e Classes Abstratas · 6 Exercícios Práticos

Nome: _____

Turma: _____

 **Encontre o erro**

 **Construa o código**

☐ **Complete o código**

 **Marque a alternativa**

PHP OO — Herança e Classes Abstratas · 1 / 9

Exercício 1 · herança · extends · parent::

Encontre e corrija os erros no código

 **Encontre o erro**

CONTEXTO

O código abaixo tenta criar uma hierarquia de animais, mas contém 3 erros. Identifique-os, marque as linhas com erro e escreva a correção no espaço abaixo.

```
<?php

class Animal {
    protected string $nome;
    protected string $som;

    public function __construct(string $nome, string $som) {
        $this->nome = $nome;
        $this->som = $som;
    }

    public function fazerSom(): string {
        return "{$this->nome} faz: {$this->som}";
    }
}

class Cachorro extend Animal {
    private string $raca;

    public function __construct(string $nome, string $raca) {
        parent::_construct($nome, 'Au au');
        $this->raca = $raca;
    }

    public function info(): string {
        return $this->fazerSom() . ' | Raça: ' . $this->raca;
    }
}

$dog = new Cachorro('Rex', 'Labrador');
echo $dog->info();
echo $dog->nome;
```

ESPAÇO PARA CORREÇÃO

Erro 1: ____ A classe Cachorro usa a palavra extend em vez de extends

Correção: __class Cachorro extends Animal _____

Erro 2: __parent::__construct(\$nome, 'Au au');

Correção: __parent::__construct(\$nome, 'Au au');

Erro 3: __echo \$dog->nome; _____

Correção: _O atributo nome é protected, portanto não pode ser acessado fora da classe. É necessário remover essa linha ou criar um método getter

SAÍDA CORRETA ESPERADA (APÓS CORREÇÕES)

Rex faz: Au au | Raça: Labrador

PHP OO — Herança e Classes Abstratas · 2 / 9

Exercício 2 · abstract class · método abstrato

Complete o código — classe abstrata Forma

☐ Complete o código

CONTEXTO

Preencha os espaços em branco (____) para que o código funcione corretamente. Cada ____ representa uma ou mais palavras.

```
<?php
__abstract____ class Forma {
    protected string $cor;

    public function __construct(string $cor) {
        $this->cor = $cor;
    }

    __abstract____ public function calcularArea(): float;

    public function descricao(): string {
        return "Forma: " . __get_class($this) ____ . " | Cor: " . $this->cor;
    }
}

class Circulo __extends____ Forma {
    public function __construct(
        private float $raio,
        string $cor
    ) {
        __parent____::__construct($cor);
    }

    public function calcularArea(): float {
        return __pi() ____ * $this->raio ** 2;
    }
}
```

```

class Retangulo extends Forma {
    public function __construct(
        private float $largura,
        private float $altura,
        string $cor
    ) {
        parent::__construct($cor);
    }

    public function calcularArea(): float {
        return $this->largura * $this->altura;
    }
}

$c = new Circulo(5.0, "azul");
$r = new Retangulo(4.0, 6.0, "vermelho");

echo $c->descricao() . ' | Área: ' . number_format($c->calcularArea(), 2) . PHP_EOL;
echo $r->descricao() . ' | Área: ' . number_format($r->calcularArea(), 2) . PHP_EOL;

```

SAÍDA ESPERADA

```

Forma: Circulo | Cor: azul | Área: 78.54
Forma: Retangulo | Cor: vermelho | Área: 24.00

```

PHP OO — Herança e Classes Abstratas · 3 / 9

Exercício 3 · herança · abstract · polimorfismo

Construa o código — sistema de funcionários

 **Construa o código**

CONTEXTO

Uma empresa tem dois tipos de funcionários: CLT (salário fixo + 13%) e Freelancer (valor por hora). Implemente as classes seguindo as regras abaixo.

REGRAS DE IMPLEMENTAÇÃO

- Classe abstrata `Funcionario` com atributos: `$nome` (string), `$departamento` (string)
- Método abstrato `calcularSalario()`: float em `Funcionario`
- Método concreto `apresentar()`: string em `Funcionario` que retorna nome + departamento + salário
- Classe `CLT` extends `Funcionario` — recebe `$salarioBase` (float); `calcularSalario()` retorna `$salarioBase * 1.0833` (13% proporcional)
- Classe `Freelancer` extends `Funcionario` — recebe `$valorHora` (float) e `$horasTrabalhadas` (int); `calcularSalario()` = `$valorHora * $horasTrabalhadas`
- Instancie um objeto de cada tipo e imprima `apresentar()`

ESPAÇO PARA DESENVOLVIMENTO

<?php

```
abstract class Funcionario {
```

```
    protected string $nome;
```

```
    protected string $departamento;
```

```
    public function __construct(string $nome, string $departamento) {
```

```
        $this->nome = $nome;
```

```
        $this->departamento = $departamento;
```

```
    }
```

```
    abstract public function calcularSalario(): float;
```

```
    public function apresentar(): string {
```

```
        return "{$this->nome} | {$this->departamento} | Salário: R$ " . number_format($this->calcularSalario(), 2, '.', '');
```

```
    }
```

```
}
```

```
class CLT extends Funcionario {
```

```
    private float $salarioBase;
```

```
    public function __construct(string $nome, string $departamento, float $salarioBase) {
```

```
        parent::__construct($nome, $departamento);
```

```
        $this->salarioBase = $salarioBase;
```

```
    }
```

```
    public function calcularSalario(): float {
```

```
        return $this->salarioBase * 1.0833; // 13ª proporcional incorporado
```

```
}
```

```
}
```

```
class Freelancer extends Funcionario {
```

```
    private float $valorHora;
```

```
    private int $horasTrabalhadas;
```

```
    public function __construct(string $nome, string $departamento, float $valorHora, int $horasTrabalhadas) {
```

```
        parent::__construct($nome, $departamento);
```

```
        $this->valorHora = $valorHora;
```

```
        $this->horasTrabalhadas = $horasTrabalhadas;
```

```
    }
```

```
    public function calcularSalario(): float {
```

```
        return $this->valorHora * $this->horasTrabalhadas;
```

```
    }
```

```
}
```

```
$ana = new CLT("Ana Silva", "RH", 5000.00);
```

```
$carlos = new Freelancer("Carlos Melo", "TI", 30.00, 160);
```

```
echo $ana->apresentar() . PHP_EOL;
```

```
echo $carlos->apresentar() . PHP_EOL;
```

SAÍDA ESPERADA

```
Ana Silva | RH | Salário: R$ 5416.50
Carlos Melo | TI | Salário: R$ 4800.00
```

PHP OO — Herança e Classes Abstratas · 4 / 9
PHP OO — Herança e Classes Abstratas · 5 / 9

Exercício 4 · abstract · herança · visibilidade

Marque a alternativa correta — conceitos teóricos

✓ Marque a alternativa

1. O que acontece se você tentar instanciar diretamente uma classe abstrata em PHP?

- ☐ a) O PHP ignora a palavra abstract e instancia normalmente.
- ☒ b) Ocorre um Fatal Error em tempo de execução.
- ☐ c) O PHP cria a instância mas desabilita os métodos abstratos.
- ☐ d) Gera apenas um aviso (Warning), não um erro fatal.

2. Analise o trecho abaixo. Qual afirmação está correta?

```
abstract class Conta {
    abstract protected function calcularJuros(): float;
    public function exibirSaldo(float $saldo): string {
        return "Saldo com juros: " . ($saldo + $this->calcularJuros());
    }
}
```

- ☐ a) A classe filha pode opcionalmente implementar calcularJuros().
- ☐ b) exibirSaldo() não pode chamar calcularJuros() pois ela é abstract.
- ☒ c) A classe filha deve implementar calcularJuros() como public ou protected, nunca private.
- ☐ d) Não é possível ter métodos concretos dentro de uma classe abstrata.

3. Qual a diferença principal entre protected e private em herança?

- ☐ a) Não há diferença; ambos ficam visíveis nas classes filhas.
- ☐ b) private é acessível na classe filha; protected apenas na classe pai.
- ☒ c) protected é acessível na classe filha e na própria classe; private só na classe onde foi declarado.
- ☐ d) protected é equivalente a public para classes do mesmo arquivo.

4. Em PHP, uma classe filha pode sobrescrever (override) um método concreto da classe pai?

- ☐ a) Não. Só métodos abstract podem ser sobrescritos.
- ☒ b) Sim, desde que o método na classe filha tenha a mesma assinatura ou mais permissiva.
- ☐ c) Sim, mas apenas se usar a palavra-chave override antes de function.
- ☐ d) Sim, mas a versão da classe pai é deletada automaticamente.

Exercício 5 · abstract · método não implementado

Encontre o erro — contrato de classe abstrata

 **Encontre o erro**

CONTEXTO

O código abaixo implementa um sistema de pagamentos. Há 2 erros conceituais/sintáticos. Identifique-os e reescreva as linhas corretas.

```
<?php
```

```
abstract class Pagamento {  
    public function __construct(  
        protected float $valor,  
        protected string $descricao  
    ) {}  
}
```

```

abstract public function processar(): string;

public function recibo(): string {
    return "Recibo: {$this->descricao} - R$ {$this->valor}";
}

}

class PagamentoCartao extends Pagamento {
    public function __construct(
        float $valor,
        string $descricao,
        private int $parcelas
    ) {
        parent::__construct($valor, $descricao);
    }

}

class PagamentoPix extends Pagamento {
    private function processar(): string {
        return "PIX processado: R$ {$this->valor}";
    }
}

$p = new PagamentoPix(150.0, "Compra online");
echo $p->processar();
echo $p->recibo();

```

ESPAÇO PARA CORREÇÃO

Erro 1: __A classe PagamentoCartao não implementou o método abstrato.

Correção: __public function processar(): string{ return "Pagamento no cartão em
{ \$this->parcelas } parcelas."; }

Erro 2: ____private function processar()

Não pode ser private.

Correção: __public function processar(): string{ return "PIX processado: R\$
{ \$this->valor}"; } _____

SAÍDA CORRETA ESPERADA

```

PIX processado: R$ 150
Recibo: Compra online - R$ 150

```

Exercício 6 · abstract · herança · override

Desafio integrado — complete e expanda

☐ Complete o código

 Construa o código

PARTE A — COMPLETE O CÓDIGO (____)

Complete os espaços em branco para que a classe Veiculo e Carro funcionem:

```
<?php

__abstract__ class Veiculo {
    public function __construct(
        protected string $marca,
        protected int $ano
    ) {}

    abstract public function __tipo__(): string;

    public function apresentar(): string {
        return $this->tipo() . ': ' . $this->marca . ' (' . $this->ano . ')';
    }

    class Carro extends Veiculo {
        public function __construct(
            string $marca,
            int $ano,
            private int $portas
        ) {
            __parent__::__construct($marca, $ano);
        }

        public function tipo(): __string__ {
            return 'Carro';
        }

        public function __apresentar__(): string {
            return parent::apresentar() . " ({ $this->portas } portas)";
        }
    }

    $c = new Carro("Toyota", 2023, 4);
    echo $c->apresentar();
}
```

PARTE B — CONSTRUA A CLASSE MOTO

Com base no padrão acima, construa a classe Moto que herda Veiculo, adiciona \$cilindrada (int) e sobrescreve apresentar() para incluir a cilindrada:

SAÍDA ESPERADA (PARTE A + PARTE B JUNTOS)

```
Carro: Toyota (2023) (4 portas)
Moto: Honda (2022) (650cc)
```